CSE 620 Project 2: Simplex

Fall 2026

Our Code

Robert Lovett John Brittian

***Abstract*—This report documents a wellthought-out and precise carrying out of the first project for CSE 620.**

***Index Terms*—heuristics, gradient descent, optimization, fun**

I. Introduction

The objective of this project is to implement and explore several variants of the gradient descent optimization algorithm. Each variant will be used to minimize each of the following functions. The only constraint is that these functions are all of the form $f : \mathbb{R}^2 \to \mathbb{R}$, so we're minimizing over the entire plane.

$$f_1(x, y) = x^2 + y^2 \tag{1}$$

$$f_2(x, y) = (1 - x)^2 + 100(y - x^2)^2 \tag{2}$$

$$f_3(x, y) = x^2 + y^2 + 10\cos x + 10\cos y \tag{3}$$

We've implemented and experimented with:

1) Vanilla Gradient Descent
2) Newton's Method (w/ Hessian)
3) AdaGrad
4) Adam (w/ $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1\text{e} - 8$)

II. Methods

Here we describe in more detail the gradient descent algorithms used, and our implementations of them.

A. Gradient Descent Algorithms

1) Vanilla Gradient Descent

This one isn't anything crazy. This algorithm simply starts with a random initial point (here we use a fixed initial point for consistency), then iteratively updates the point using the gradient of f:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

This algorithm in pseudocode looks like this:

Initialize $(x, y) \leftarrow (x_0, y_0)$
Initialize $\alpha \leftarrow \alpha_0$
for $t = 1, \ldots, T$ **do**
    $(x, y) \leftarrow (x, y) - \alpha \nabla f(x, y)$
end for
return (x, y)

2) Newton's Method (w/ Hessian)

Newton's Method is a second-order optimization algorithm that uses both the gradient and the Hessian matrix of f:

$$\boldsymbol{H_f} = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix}$$

This algorithm in pseudocode looks like this:

Initialize $(x, y) \leftarrow (x_0, y_0)$
Initialize $\alpha \leftarrow \alpha_0$
for $t = 1, \ldots, T$ **do**
    $(x, y) \leftarrow (x, y) - \alpha H_f^{-1}(x, y) \nabla f(x, y)$
end for
return (x, y)

3) AdaGrad

AdaGrad is an adaptive learning rate optimization algorithm that adjusts the learning rate for each parameter based on the historical gradients. It accumulates the squared gradients and uses them to scale the learning rate. Some implementations the entire outer product of past gradients with itself, but ours uses only the diagonal of that matrix, which is much more computationally efficient.

This algorithm in pseudocode looks like this:

Initialize $(x, y) \leftarrow (x_0, y_0)$
Initialize $\alpha \leftarrow \alpha_0$
Initialize $g_{acc} \leftarrow (0, 0)$
for $t = 1, \ldots, T$ **do**
    $g \leftarrow \nabla f(x, y)$
    $g_{acc} \leftarrow g_{acc} + g \odot g$
    $(x, y) \leftarrow (x, y) - \alpha \cdot (g_{acc})^{\odot -\frac{1}{2}} \cdot \nabla f(x, y)$
end for
return (x, y)

where $\odot$ denotes the Hadamard operations.

4) Adam

Adam is essentially an intelligent combination of AdaGrad with momentum. It tracks an exponentially decaying average of past gradients and squared gradients, and uses these to adapt the learning rate for each parameter. In this project we use the following values for Adam's unique hyperparameters:

$$\beta_1 = 0.9 \qquad \beta_2 = 0.999 \qquad \epsilon = 1\text{e} - 8$$

This algorithm in pseudocode looks like this:

Initialize $(x, y) \leftarrow (x_0, y_0)$
Initialize $\alpha \leftarrow \alpha_0$
Initialize $m \leftarrow (0, 0)$
Initialize $v \leftarrow (0, 0)$
for $t = 1, \ldots, T$ **do**
  $g \leftarrow \nabla f(x, y)$
  $m \leftarrow \beta_1 m + (1 - \beta_1) g$
  $v \leftarrow \beta_2 v + (1 - \beta_2) (g \odot g)$
  $\hat{m} \leftarrow \frac{m}{1 - \beta_1^t}$
  $\hat{v} \leftarrow \frac{v}{1 - \beta_2^t}$
  $(x, y) \leftarrow (x, y) - \alpha \cdot \left(\hat{m} \oslash \left(\sqrt{\hat{v}} + \epsilon \right) \right)$
end for
return (x, y)

where $\oslash$ denotes the Hadamard division.

B. Our Code

We wrote our scripts in Python and built the algorithms from (mostly) scratch. They can be found in the appendix, findable from these links - Gradient Descent, Newton's Method, AdaGrad, and Adam.

For running the code, first clone the repository. Next, simply running the script `run_all.py` will generate all the results, plots, and data files.

III. Results

Running our code generated the following plots for each combination of algorithm and function. Furthermore, we have complete, tabular results for each hyperparameter combination.

A. Adagrad

1) Cosine Bumps



Fig. 1. Adagrad on the Cosine Bumps function

2) Quadratic

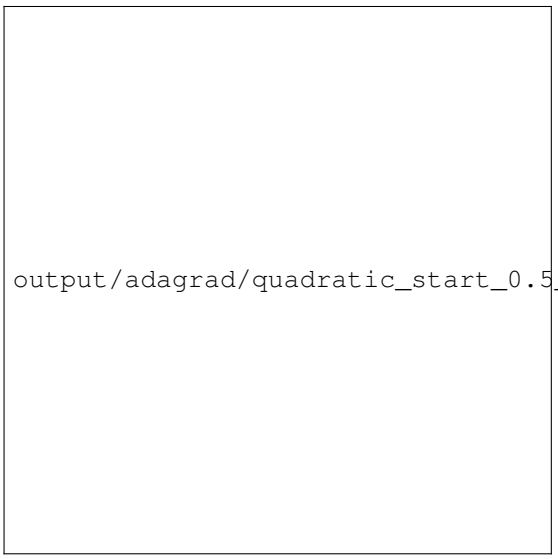


Fig. 2. Adagrad on the Quadratic function

3) Rosenbrock

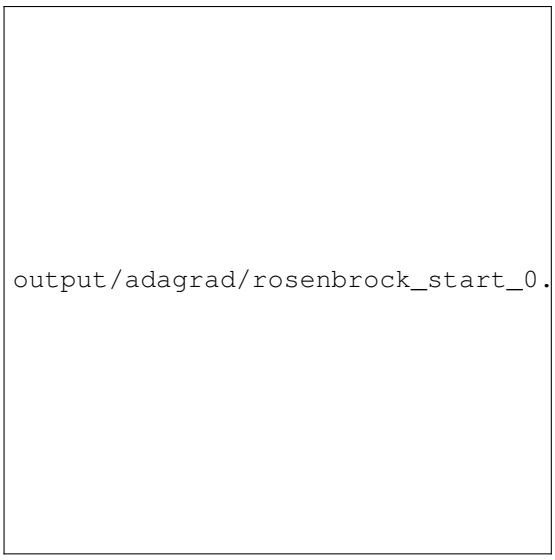


Fig. 3. Adagrad on the Rosenbrock function

TABLE I
Adagrad results for $(x_0, y_0) = (-2, 2)$.

Function	LR	Time (s)	Iterations	Final f(x,y)
Quadratic	0.001	0.01185	2000	7.317
	0.01	0.01267	2000	2.864
	0.1	0.007218	1098	1.252×10^{-8}
Rosenbrock	0.001	0.01255	2000	260.4
	0.01	0.01238	2000	6.745
	0.1	0.01226	2000	6.041
Cosine Bumps	0.001	0.01229	2000	-1.153
	0.01	0.01329	2000	-3.605
	0.1	0.0007854	117	-3.618

TABLE II
ADAGRAD RESULTS FOR $(x_0, y_0) = (0.5, -1.5)$.

Function	LR	Time (s)	Iterations	Final f(x,y)
Quadratic	0.001	0.01215	2000	2.168
	0.01	0.01176	2000	0.5288
	0.1	0.003649	635	4.164×10^{-9}
Rosenbrock	0.001	0.01201	2000	252.1
	0.01	0.01189	2000	56.78
	0.1	0.01197	2000	0.4182
Cosine Bumps	0.001	0.01224	2000	11.01
	0.01	0.01255	2000	1.910
	0.1	0.003486	568	-3.618

TABLE III
ADAGRAD RESULTS FOR $(x_0, y_0) = (3, 3)$.

Function	LR	Time (s)	Iterations	Final f(x,y)
Quadratic	0.001	0.01161	2000	16.96
	0.01	0.01251	2000	9.407
	0.1	0.01194	2000	5.987×10^{-7}
Rosenbrock	0.001	0.01216	2000	2925.2
	0.01	0.01200	2000	307.7
	0.1	0.01240	2000	1033.7
Cosine Bumps	0.001	0.01247	2000	-2.489
	0.01	0.01263	2000	-3.618
	0.1	0.0004805	74	-3.618

B. ADAM

1) Cosine Bumps



Fig. 4. ADAM on the Cosine Bumps function

2) Quadratic



Fig. 5. ADAM on the Quadratic function

3) Rosenbrock

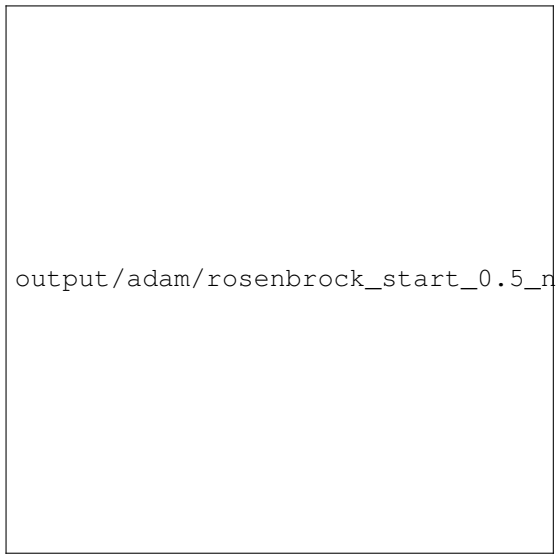


Fig. 6. ADAM on the Rosenbrock function

TABLE IV
ADAM RESULTS FOR $(x_0, y_0) = (-2, 2)$.

Function	LR	Time (s)	Iterations	Final f(x,y)
Quadratic	0.001	0.01757	2000	0.4831
	0.01	0.008406	796	1.545×10^{-9}
	0.1	0.001583	181	5.123×10^{-8}
Rosenbrock	0.001	0.01945	2000	6.571
	0.01	0.01893	2000	5.547
	0.1	0.01836	2000	0.001933
Cosine Bumps	0.001	0.01738	1860	-3.618
	0.01	0.001654	180	-3.618
	0.1	0.001844	193	-3.618

TABLE V
ADAM RESULTS FOR $(x_0, y_0) = (0.5, -1.5)$.

Function	LR	Time (s)	Iterations	Final f(x,y)
Quadratic	0.001	0.01749	2000	0.03365
	0.01	0.005208	569	7.937×10^{-10}
	0.1	0.002129	230	7.964×10^{-11}
Rosenbrock	0.001	0.01837	2000	5.207
	0.01	0.01894	2000	0.2454
	0.1	0.01872	2000	6.877×10^{-6}
Cosine Bumps	0.001	0.01952	2000	-3.387
	0.01	0.004131	436	-3.618
	0.1	0.002267	242	-3.618

TABLE VI
ADAM RESULTS FOR $(x_0, y_0) = (3, 3)$.

Function	LR	Time (s)	Iterations	Final f(x,y)
Quadratic	0.001	0.01773	2000	3.411
	0.01	0.01042	1202	3.622×10^{-9}
	0.1	0.001952	215	4.328×10^{-9}
Rosenbrock	0.001	0.01836	2000	6.795
	0.01	0.01797	2000	1.022
	0.1	0.01848	2000	0.4296
Cosine Bumps	0.001	0.01411	1434	-3.618
	0.01	0.001653	172	-3.618
	0.1	0.001794	193	-3.618

C. Gradient Descent

1) Cosine Bumps

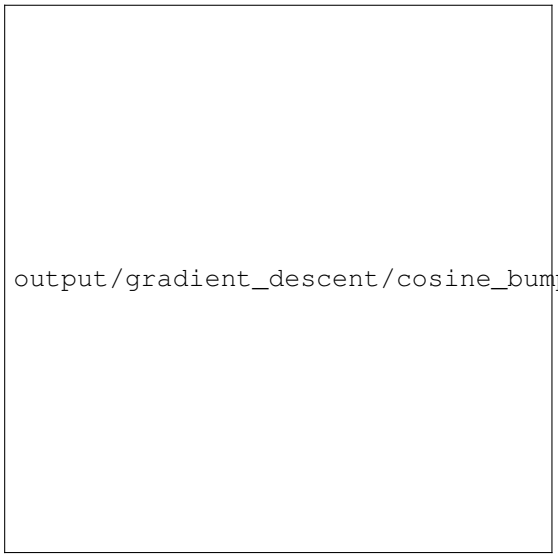


Fig. 7. Gradient Descent on the Cosine Bumps function

2) Quadratic

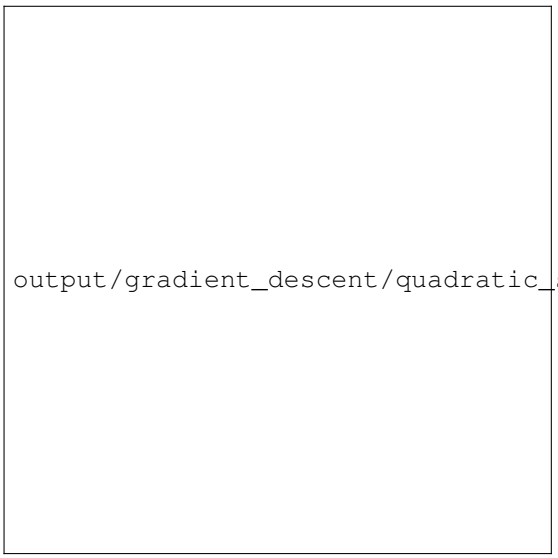


Fig. 8. Gradient Descent on the Quadratic function

3) Rosenbrock



Fig. 9. Gradient Descent on the Rosenbrock function

TABLE VII
GRADIENT DESCENT RESULTS FOR $(x_0, y_0) = (-2, 2)$.

Function	LR	Time (s)	Iterations	Final f(x,y)
Quadratic	0.001	0.008853	2000	0.002662
	0.01	0.002769	543	2.369×10^{-9}
	0.1	0.0002992	61	1.202×10^{-11}
Rosenbrock	0.001	0.009413	2000	0.1283
	0.01	0.0003998	6	NaN
	0.1	0.009050	6	NaN
Cosine Bumps	0.001	0.004102	877	-3.618
	0.01	0.0005262	106	-3.618
	0.1	0.00007510	6	-3.618

TABLE VIII
GRADIENT DESCENT RESULTS FOR $(x_0, y_0) = (0.5, -1.5)$.

Function	LR	Time (s)	Iterations	Final f(x,y)
Quadratic	0.001	0.008413	2000	0.0008320
	0.01	0.002254	514	2.390×10^{-9}
	0.1	0.0002855	58	1.433×10^{-11}
Rosenbrock	0.001	0.009246	2000	0.03175
	0.01	0.00008010	7	NaN
	0.1	0.00005880	6	NaN
Cosine Bumps	0.001	0.005419	1096	-3.618
	0.01	0.0006307	128	-3.618
	0.1	0.00008390	9	-3.618

TABLE IX
GRADIENT DESCENT RESULTS FOR $(x_0, y_0) = (3, 3)$.

Function	LR	Time (s)	Iterations	Final f(x,y)
Quadratic	0.001	0.008602	2000	0.005990
	0.01	0.002520	563	2.376×10^{-9}
	0.1	0.0003094	63	1.108×10^{-11}
Rosenbrock	0.001	0.00005010	8	NaN
	0.01	0.00003760	6	NaN
	0.1	0.00005870	6	NaN
Cosine Bumps	0.001	0.004042	814	-3.618
	0.01	0.0005418	99	-3.618
	0.1	0.00007260	6	-3.618

D. Newton

1) Cosine Bumps

output/newton/cosine_bumps_start_0.5_neg1.5_lr_0.5.png

Fig. 10. Newton's Method on the Cosine Bumps function

2) Quadratic

output/newton/quadratic_start_0.5_neg1.5_lr_0.

Fig. 11. Newton's Method on the Quadratic function

3) Rosenbrock

output/newton/rosenbrock_start_0.5_neg1.5_lr_0

Fig. 12. Newton's Method on the Rosenbrock function

TABLE X
NEWTON RESULTS FOR $(x_0, y_0) = (-2, 2)$.

Function	LR	Time (s)	Iterations	Final f(x,y)
Quadratic	0.25	0.0005381	48	8.109×10^{-12}
	0.5	0.0003780	22	4.547×10^{-13}
	1.0	0.00008460	2	0
Rosenbrock	0.25	0.0009911	102	1.791×10^{-12}
	0.5	0.0005581	54	1.540×10^{-13}
	1.0	0.0001111	6	0
Cosine Bumps	0.25	0.0004562	43	-3.618
	0.5	0.0002393	19	-3.618
	1.0	0.0001156	5	-3.618

TABLE XI
NEWTON RESULTS FOR $(x_0, y_0) = (0.5, -1.5)$.

Function	LR	Time (s)	Iterations	Final f(x,y)
Quadratic	0.25	0.0005968	46	8.009×10^{-12}
	0.5	0.0002286	21	5.684×10^{-13}
	1.0	0.00007750	2	0
Rosenbrock	0.25	0.0007514	70	1.749×10^{-12}
	0.5	0.0003686	34	9.316×10^{-14}
	1.0	0.0001111	5	0
Cosine Bumps	0.25	0.0004815	43	8.191
	0.5	0.0002492	21	8.191
	1.0	0.001856	198	8.191

TABLE XII
NEWTON RESULTS FOR $(x_0, y_0) = (3, 3)$.

Function	LR	Time (s)	Iterations	Final f(x,y)
Quadratic	0.25	0.0004530	50	5.773×10^{-12}
	0.5	0.0002949	23	2.558×10^{-13}
	1.0	0.00007740	2	0
Rosenbrock	0.25	0.0009182	95	1.789×10^{-12}
	0.5	0.0004774	49	1.037×10^{-13}
	1.0	0.0001174	6	0
Cosine Bumps	0.25	0.0005216	43	-3.618
	0.5	0.0002938	20	-3.618
	1.0	0.00009760	4	-3.618

IV. EXPERIMENTAL ANALYSIS

Looking at the plots from the previous Results, we can see that the Gradient Descent algorithm produces the simplest pathes through the contours of each function, with little to no oscillation. In fact, despite being the simplest algorithm, it tends to outperform the others when looking at the number of iterations to convergence and final function value.

Looking at the Rosenbrock function in Tables IV, V, and VI, we see that Adam's performance is extremely variable, which is to be expected with the Rosenbrock function's peculiar nature.

Furthermore, we can see that, for the Adagrad and Newton algorithms on the Cosine Bumps function, the initial point is very important and can lead to vastly different end results.

Overall, it looks like the Newton's Method algorithm converges in the least amount of iterations, whilst Adagrad takes the most amount of iterations. In terms of runtime, Gradient Descent and Newton's methods are about an order of magnitude or two faster than Adam and Adagrad.

V. CONCLUSION

In conclusion, our findings are mostly in line with our expectations. Gradient Descent is the simplest algorithm, and therefore the fastest. We notice that it tends to perform the best across the other algorithms - perhaps since the functions tested aren't too nonconvex. However, Gradient Descent gave poor results on the Rosenbrock function, which could be attributed to potential overflow in the gradient calculations.

Newton's Method had the fastest raw convergence numbers, but saw greater variability in the Cosine Bumps function based off the initial starting point. This is a potentially acceptable tradeoff.

Adagrad's performance was mostly non-remarkable. With the right hyeprparameters, it found good solutions for the Quadratic and Cosine Bumps functions, but it struggled with Rosenbrock. Perhaps its advantages lie in its computational efficiency compared to algorithms like Adam, however computational efficiency was of little concern in these experiments.

Adam could best be described as the Goldilocks of the algorithms we tested, achieving solid final results consistently, albeit more slowly than Gradient Descent or Newton's Method. Its trajectories show us that it is more robust to the initial starting point than Newton's Method, but that it still slightly struggles with Rosenbrock.

In order to further detect the ideal use cases of each algorithm, we could believe it would be beneficial to experiment with more wacky (nonconvex) functions to see how the complex algorithms perform, such as the Schwefel function. We hypothesize that the bells and whistles of the more complex algorithms aren't as useful for the functions tested here, and that Adagrad and Adam would prove more useful in larger neural network type applications.

ACKNOWLEDGMENT

We would like to thank and acknowledge our professor Dr. Olfa Nasraoui for her guidance and support throughout this project.

APPENDIX

Listing 1. Gradient Descent implementation

```
def gradient_descent(func, grad, start, learning_rate):
    x = start.astype(float).copy()
    path = [x.copy()]

    start_time = time.perf_counter()
    converged = False

    for iteration in range(1, MAX_ITER + 1):
        new_x = x - learning_rate * grad(x)
        path.append(new_x.copy())

        if not np.all(np.isfinite(new_x)):
            x = new_x
            break

        if np.linalg.norm(new_x - x) < TOL:
            x = new_x
            converged = True
            break

        x = new_x

    runtime = time.perf_counter() - start_time
    return x, path, iteration, converged, runtime
```

Listing 2. Newton's Method implementation

```
def newtons_method(func, grad, hess, start, damping):
    x = start.astype(float).copy()
    path = [x.copy()]

    start_time = time.perf_counter()
    converged = False

    for iteration in range(1, MAX_ITER + 1):
        g = grad(x)
        H = hess(x)

        try:
            step = np.linalg.solve(H, g)
        except np.linalg.LinAlgError:
            # Pseudoinverse allows the experiment to continue if Hessian is singular.
            step = np.linalg.pinv(H) @ g

        new_x = x - damping * step
        path.append(new_x.copy())

        if not np.all(np.isfinite(new_x)):
            x = new_x
            break

        if np.linalg.norm(new_x - x) < TOL:
            x = new_x
            converged = True
```

```
            break

        x = new_x

    runtime = time.perf_counter() - start_time
    return x, path, iteration, converged, runtime
```

Listing 3. AdaGrad implementation

```
def adagrad(func, grad, start, learning_rate, epsilon=1e-8):
    x = start.astype(float).copy()
    accumulated_sq_grad = np.zeros_like(x)
    path = [x.copy()]

    start_time = time.perf_counter()
    converged = False

    for iteration in range(1, MAX_ITER + 1):
        g = grad(x)
        accumulated_sq_grad += g * g
        adjusted_step = learning_rate * g / (np.sqrt(accumulated_sq_grad) + epsilon)
        new_x = x - adjusted_step
        path.append(new_x.copy())

        if not np.all(np.isfinite(new_x)):
            x = new_x
            break

        if np.linalg.norm(new_x - x) < TOL:
            x = new_x
            converged = True
            break

        x = new_x

    runtime = time.perf_counter() - start_time
    return x, path, iteration, converged, runtime
```

Listing 4. Adam implementation

```
def adam(func, grad, start, learning_rate,
         beta1=0.9, beta2=0.999, epsilon=1e-8):
    x = start.astype(float).copy()
    m = np.zeros_like(x)
    v = np.zeros_like(x)
    path = [x.copy()]

    start_time = time.perf_counter()
    converged = False

    for iteration in range(1, MAX_ITER + 1):
        g = grad(x)

        m = beta1 * m + (1 - beta1) * g
        v = beta2 * v + (1 - beta2) * (g * g)

        m_hat = m / (1 - beta1 ** iteration)
        v_hat = v / (1 - beta2 ** iteration)
```

```
        new_x = x - learning_rate * m_hat / (np.sqrt(v_hat) + epsilon)
        path.append(new_x.copy())

        if not np.all(np.isfinite(new_x)):
            x = new_x
            break

        if np.linalg.norm(new_x - x) < TOL:
            x = new_x
            converged = True
            break

        x = new_x

    runtime = time.perf_counter() - start_time
    return x, path, iteration, converged, runtime
```